Replication package

A Matching-Theoretic Approach to Optimal Charging Station Selection for Electric Vehicles

Luca Ambrosino, Hong T. M. Chu, Laurent El Ghaoui, Giuseppe C. Calafiore

October 14, 2026

1 What this is

Everything needed to reproduce the figures and tables of the paper: the MATLAB code, the charging-network data, and the scripts that built those data from the public source. Each experiment is one function, and each function states in its header which section of the paper it produces.

The code is written to be run and read, not only run. Where a choice is not obvious, or where we got it wrong once, the reason is in a comment next to it.

2 Requirements

MATLAB R2021b or later. The experiments were run on R2023b under Windows, on a machine with 14 physical cores; the timings reported in Section 3.4.2 of the paper refer to that machine.

Three toolboxes are used:

- **Statistics and Machine Learning Toolbox**, for `datasample` and `quantile`;
- **Mapping Toolbox**, for the basemaps under the city maps. Without it the maps still draw, on plain axes;
- **Optimization Toolbox** or **Mosek**, for the MILP and its relaxation in `time_scaling`. Mosek is preferred and is what the paper used; `intlinprog` is used automatically if `mosekopt` is not on the path.

Nothing else is needed. The Hungarian algorithm, the Hopcroft–Karp matching and the greedy policies are all included here and are self-contained.

3 Layout

`code/`	the nine experiments and everything they call
`data/`	the three city networks, plus the national source file
`data_preparation/`	how the city files were built from the source
`figures/`	the figures, exactly as the code produces them

4 Running everything

From the `code/` folder:

```
>> run_all_simulations
```

This runs the nine experiments in the order in which they appear in the paper and takes a couple of hours, most of it in `time_scaling`, which solves a mixed-integer program at eight sizes, and in the three real networks, which solve 100 assignment problems each.

Order matters in one place only. `calibration_pareto` writes `calibrated_alpha.mat`, and every later script reads the two coefficients from it. A copy of that file is shipped with the package, so the calibration step can be skipped and the numbers in the paper still come out. Each script announces, once per session, which coefficients it is using and where they came from; if it says *literature* instead of *calibrated*, the `.mat` file is not on the path.

5 Running one experiment

Paper	Command	Produces
Sec. 3.2	`calibration_pareto`	Table 1, Fig. 3
Sec. 3.3.1, 3.3.3	`structural_analysis`	Fig. 4, Fig. 5
Sec. 3.3.2	`synthetic_comparison`	Table 2
Sec. 3.4.1	`toy_equivalence`	Fig. 6
Sec. 3.4.2	`time_scaling`	Fig. 7
Sec. 4.1	`realworld_analysis(...)`	Tables 3–4, Figs. 8–11
Sec. 4.2	`feasibility_sweep(...)`	Table 5, Fig. 12
Sec. 4.3	`load_sweep(...)`	Table 6, Fig. 13
Sec. 4.4	`sensitivity_alpha(...)`	Fig. 14

The four scripts that take an argument want the path to a city file:

```
>> realworld_analysis('../data/CS_location_DC_complete.csv')
>> realworld_analysis('../data/CS_location_Chicago.xlsx')
>> realworld_analysis('../data/CS_location_Manhattan.xlsx')
>> feasibility_sweep('../data/CS_location_DC_complete.csv')
>> load_sweep('../data/CS_location_DC_complete.csv')
>> sensitivity_alpha('../data/CS_location_DC_complete.csv')
```

The city name is inferred from the file name and used to tag the figures, so the three cities do not overwrite each other. Pass it explicitly as a third argument if the file is named differently.

The code draws more than the paper shows. Every city produces its own maps and its own boxplot of each criterion, and `synthetic_comparison` draws the boxplots behind Table 2, whereas the paper displays one city per figure to keep the page count down; `figures/` therefore holds rather more files than there are figures in the paper. The two bipartite-graph diagrams of Section 2.3, Figures 1 and 2, are the only ones the code does not generate: they were drawn by hand.

Every script prints its results table to the console in a form that can be pasted straight into LaTeX. The tables in the paper are those blocks, merged across cities by hand.

6 The data

`data/` holds the three networks used in Section 4:

City	File	Stations	Ports
Washington DC	`CS_location_DC_complete.csv`	334	1053
Chicago	`CS_location_Chicago.xlsx`	277	725
Manhattan (NY)	`CS_location_Manhattan.xlsx`	352	1061

Each row is one station, with its coordinates and the number of connectors of each type. The code expands every station into its individual ports, since the model is formulated at port level.

`USA_EV_CS.csv` is the national source these were extracted from. It is included so that the three city files can be rebuilt, or new cities added, without hunting for the original download.

7 Rebuilding the city files

The source file gives addresses, not coordinates. `data_preparation/` contains the pipeline that produced the three city files, in the order in which it is run:

1. `process_EV_charging_data` selects the rows of one metropolitan area from the national file;
2. `Coordinates_Finder` sends the addresses to the U.S. Census Bureau batch geocoder, in blocks, and collects the answers;
3. `merge_census_results` joins the geocoded coordinates back onto the station records;
4. `clean_city_file` drops the rows the geocoder could not resolve and writes the final file.

Step 2 needs an internet connection and is the slow one. There is no need to run any of this to reproduce the paper: the three output files are already in `data/`.

8 What is random and what is not

The physical network is fixed: station coordinates and connector powers come from the data and never change. Everything describing the demand and the operating conditions is redrawn at every instance, namely vehicle positions, energy demands, budgets, tariffs and waiting times.

Every script seeds the generator at the top, so a given script reproduces its own numbers exactly. Across versions of MATLAB the stream may differ in the last digits; the figures in the paper are averages over 100 instances and are stable well beyond that.

One consequence worth knowing: adding a method to a comparison consumes random numbers and shifts the stream, so the other methods move slightly too. If a number changes in the second decimal after you edit a script, this is usually why.

9 A note on the big-M

Forbidden pairs are given to the Hungarian algorithm as a large but *finite* penalty. On an infeasible instance the algorithm therefore does not fail: it returns a complete assignment that quietly uses forbidden pairs. Counting the assigned vehicles detects nothing, since the count is N either way.

Every script that solves an assignment problem checks feasibility with `hopcroft_karp` before solving, and verifies afterwards that the returned assignment lies inside the acceptable matrix. If you reuse this code, keep both checks.

10 License and contact

The code is released under the MIT license. The charging-station data are derived from a public U.S. Department of Energy dataset and are redistributed here for reproducibility only.

Questions and bug reports: luca.ambrosino@polito.it.